AUTOMATIC BACKGROUND REMOVAL AND COMPARATIVE ANALYSIS OF MULTIPLE SEGMENTATION MODELS

ASSIST. PROF. DR. SEMRA KOCAASLAN KARAMZADEH

Hüseyin KAYA (220303036) &

Ada Şevval SARİ (220303023)

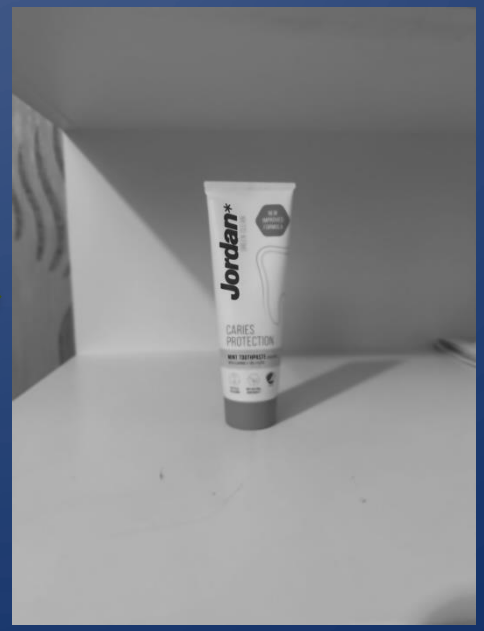
PROJECT SCOPE & OBJECTIVES

- **Problem Definition:** Automating foreground extraction of e-commerce products from complex backgrounds to replace manual segmentation.
- **Integrated Pipeline:** Seamlessly combining **Spatial Domain Filtering** (Bilateral Filter), **Morphological Operations** (Opening/Closing), and **Image Segmentation**.
- **Core Methodology:** Comparative performance analysis of six distinct traditional segmentation techniques:
  - **Thresholding:** Otsu and Adaptive Thresholding.
  - **Edge-Based:** Canny Edge Detection with boundary filling.
  - **Clustering-Based:** K-Means Clustering for color space segmentation.
  - **Region-Based:** Watershed Segmentation.
  - **Graph-Based:** GrabCut algorithm with ROI (Region of Interest).
- **Evaluation Metrics:** Quantitative validation using **Accuracy, Precision, Recall, F1-Score, IoU,** and **MSE**.
- **Implementation:** Developed and tested using **Python/OpenCV** with real-world custom datasets

DATA ACQUISITION AND INTEGRATED PRE-PROCESSING PIPELINE

- **Custom Dataset:** 6 high-resolution images of "Jordan Toothpaste" captured under various angles and non-uniform lighting conditions.
- **Noise Reduction (Spatial Domain Filtering):** Applied **Bilateral Filtering** to suppress Gaussian sensor noise.
- **Edge Preservation:** Unlike Gaussian blur, Bilateral filtering was chosen to maintain sharp object boundaries, which is critical for the subsequent segmentation stage.
- **Ground Truth & Alignment:** Manual binary masks were created and verified using **Image Overlay** (Alpha Blending) to ensure 100% pixel-wise registration accuracy.

Original image

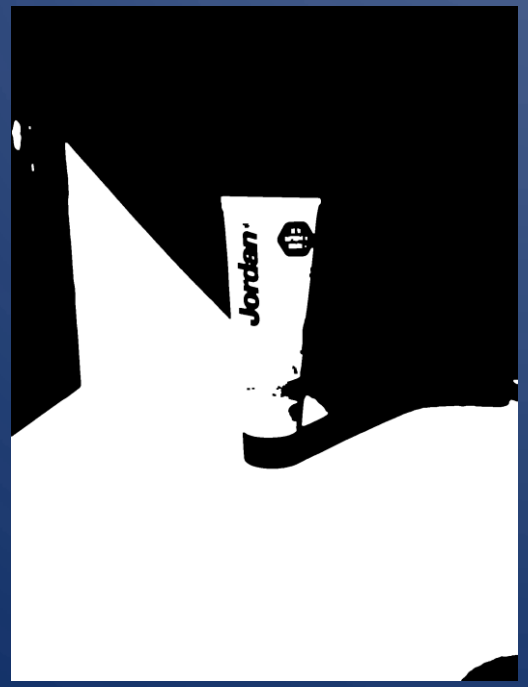


Denoised

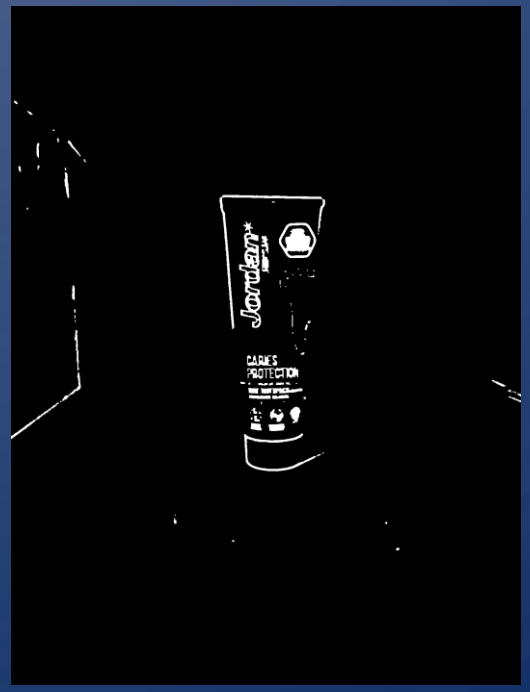
```
# 2. Convert to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 3. Apply Bilateral Filter (Spatial Domain Filtering)
# Cleans noise while preserving edges.
denoised = cv2.bilateralFilter(gray, d=15, sigmaColor=75, sigmaSpace=75)

# 4. PHYSICAL SAVING (Most critical step of the pipeline)
# Saves the image to the 'denoised_images' folder.
save_path = f"{output_folder}/{num}_denoised.png"
cv2.imwrite(save_path, denoised)
print(f"Success: {save_path} saved.")
```

INTENSITY-BASED SEGMENTATION: OTSU VS. ADAPTIVE THRESHOLDING

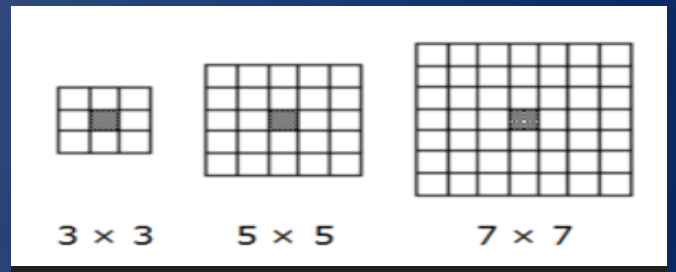
- **Otsu's Method:** Automatically calculates the optimal threshold value by minimizing intra-class variance. Effective for bimodal histograms.
- **Adaptive Thresholding:** Calculates thresholds for smaller regions to overcome non-uniform lighting and shadows.
- **Post-processing:** Applied **Morphological Operations** (Opening & Closing) to fill gaps within the object and remove background salt-and-pepper noise.
- **Findings:** Otsu showed high **Recall (0.81)** but struggled with precision due to shadows, whereas Adaptive managed local variations better but produced more fragmented masks



Otsu thresholding
(High recall)



Adaptive Thresholding
(Shadow Compensation)



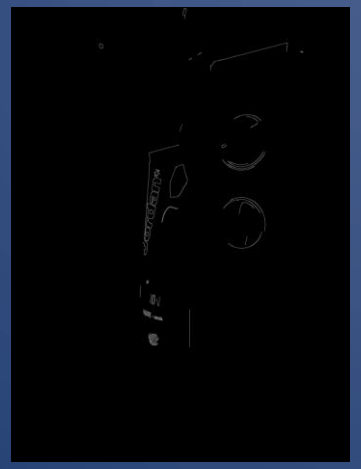

```
# --- STEP 1: ADAPTIVE SEGMENTATION ---
# Local Thresholding with Gaussian Weighting to overcome shadows
thresh = cv2.adaptiveThreshold(denoised, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY_INV, 11, 2)

# --- STEP 2: MORPHOLOGICAL REFINEMENT ---
kernel = np.ones((3,3), np.uint8)
# Closing: Fills small holes inside the object (2 iterations)
morph = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)
# Opening: Removes small background noise pixels
final_mask = cv2.morphologyEx(morph, cv2.MORPH_OPEN, kernel, iterations=1)

# --- STEP 3: PHYSICAL SAVING ---
cv2.imwrite(f"{num}_algo_result_adaptive.png", final_mask)
```

EDGE-BASED STRATEGY: FROM CANNY EDGES TO SOLID MASKS

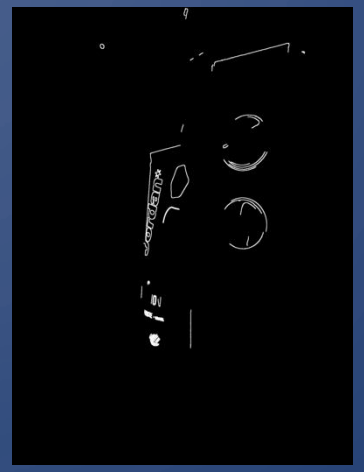
- **Canny Edge Detection:** Detected structural boundaries using double thresholding and hysteresis.
- **Dilation Step:** Used morphological dilation to bridge gaps between fragmented edge pixels.
- **Contour Analysis:** Identified the external boundary of the toothpaste tube using *findContours.*
- **Filling Process:** Applied *drawContours* with *FILLED* thickness to transform hollow edges into a solid segmentation mask.

** **Low contrast** between the white product and background caused Canny to only detect **high-gradient areas** like the logo. This failure demonstrates the limits of edge-based methods in low-contrast scenes.*

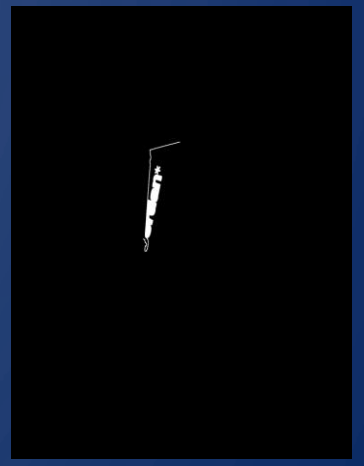
PIPELINE FLOW



1.Canny Edge Detection

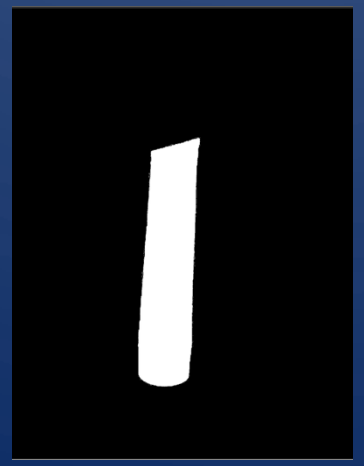


2.Morphological Dilation



3.Contour Filling (Final Mask)

Original image



Ideal Ground Truth

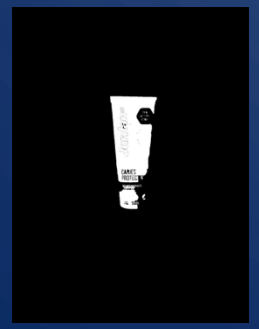
REGION-BASED SEGMENTATION STRATEGIES

GrabCut Algorithm (Iterative Segmentation)

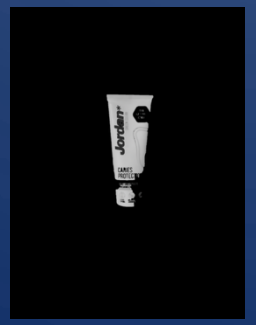
Core Logic: Uses ***Gaussian Mixture Models (GMM)*** to estimate foreground/background color distributions.

- **Mechanism:** User-defined **Bounding Box** + Iterative Refinement.
- **Advantage:** Superior handling of **complex backgrounds** and fine textures.

Original + ROI



GrabCut mask

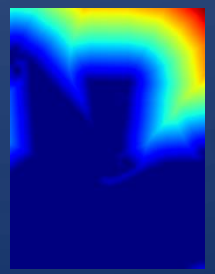


Final result

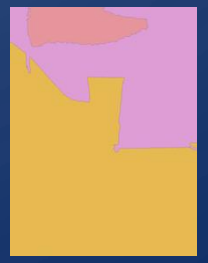
Watershed Transform (Marker-Controlled)

Core Logic: Interprets pixel intensities as topographical surface: peaks (high intensity) and valleys (low intensity). It simulates ***flooding*** the valleys.

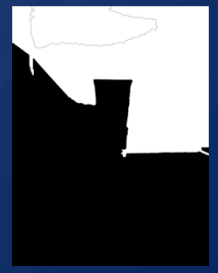
- **Mechanism:** Uses **Markers** and **Flooding** to define boundaries at watershed lines.
- **Advantage:** Best for separating **touching objects** and defining precise boundaries.



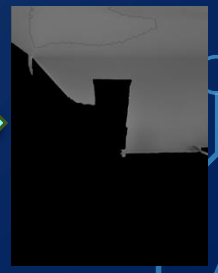
Distance Transform



Watershed maskers



Watershed mask



Final result

CLUSTERING-BASED STRATEGIES: K-MEANS CLUSTERING (UNSUPERVISED LEARNING)

Unsupervised Learning: Partitions the image into **K clusters** by grouping pixels with similar color features.

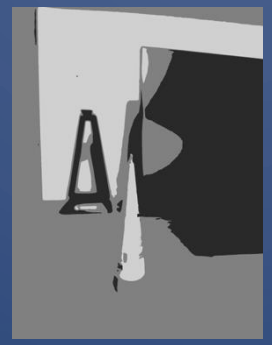
Color Space Analysis: Operates in **RGB or HSV** space to identify distinct color distributions.

Centroid Optimization: Iteratively minimizes the **Squared Euclidean Distance** between pixels and cluster centers.

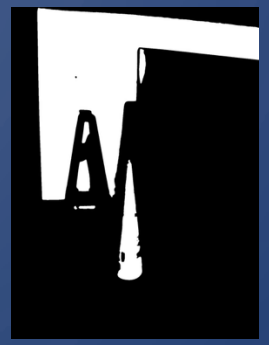
Automatic Selection: The final mask is generated by isolating the cluster with the highest **luminance** (white product body).

```python
# STEP 1: K-Means Configuration
# Define criteria (Epsilon and Max Iterations) and set K=3 (Product, Background, Shadows)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
K = 3
ret, label, center = cv2.kmeans(data, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

# STEP 2: Automatic Cluster Selection (Engineering Solution)
# Calculate luminance for each cluster center to identify the brightest group (the product)
# Formula: Y = 0.299*R + 0.587*G + 0.114*B
brightness = [0.299*c[2] + 0.587*c[1] + 0.114*c[0] for c in center]
target_label = np.argmax(brightness)

# STEP 3: Mask Generation
# Reshape labels back to original image dimensions
label_image = label.reshape(img_color.shape[:2])
# Convert target cluster to white (255) and others to black (0)
mask = (label_image == target_label).astype(np.uint8) * 255

# STEP 4: Post-processing (Noise Reduction)
# Apply Morphological Opening to remove small artifacts and outliers
kernel = np.ones((5,5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# STEP 5: Final Foreground Extraction
# Apply bitwise-AND using the generated mask to extract the colored product
result_extraction = cv2.bitwise_and(img_color, img_color, mask=mask)
```

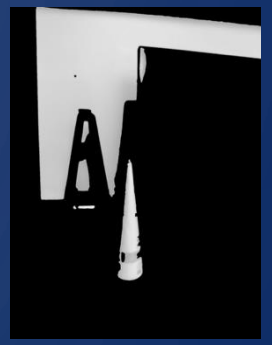
PIPELINE FLOW



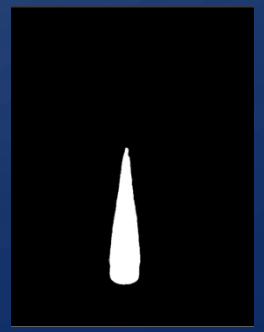
K-Means clusters



K-Means mask



Final result



Ideal Ground Truth

Original image

MATHEMATICAL FOUNDATIONS OF SEGMENTATION

- **Otsu's Method:** Minimizes intra-class variance $(\sigma_w^2)$by finding an optimal threshold (t) that separates two classes (pixels).

$$\sigma_w^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$$

- **K-Means Clustering:** Minimizes the **Squared Euclidean Distance** (J) between pixels $(x_i)$and their assigned cluster centroids (μ_j).

$$J = \sum_{j=1}^{K}\sum_{i=1}^{n}\left|\left|x_i - \mu_j\right|\right|^2$$

- **Canny Edge Detection:** Uses **Gaussian Derivatives** to find the image gradient magnitude (G) and direction (θ).

$$G = \sqrt{G_x^2 + G_y^2}$$

- **Adaptive Thresholding:** Calculates a local threshold $T(x, y)$ for each pixel based on the mean of its $n \times n$ neighborhood minus a constant C.

$$T(x, y) = \text{mean}(n \times n \text{ neighborhood}) - C$$

- **Watershed Transform:** Interprets the image gradient magnitude as a topographic surface; it minimizes the "immersion cost" by finding catchment basins through the **Distance Transform** (D).

$$D(p) = \min_{q \in \text{Background}} \text{dist}\,(p, q)$$

- **GrabCut (Graph-Cut):** Based on energy minimization of a **Gibbs Energy** function E,which balances data coherence (color GMMs) and spatial smoothness (edge contrast).

$$E(\alpha, k, \theta, z) = U(\alpha, k, \theta, z) + V(\alpha, z)$$

QUANTITATIVE EVALUATION METRICS

- Intersection over Union (IoU) / Jaccard Index: Measures the ratio of the overlap between the Predicted Mask and the Ground Truth to their union area.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{TP}{TP + FP + FN}$$

- Pixel Accuracy: Represents the percentage of correctly classified pixels (both object and background) in the total image.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision & Recall:**
  - **Precision:** The ability to avoid labeling background pixels as foreground.
  - **Recall:** The ability to capture all relevant foreground pixels

$$\text{Precision} = \frac{TP}{TP + FP} \quad , \quad \text{Recall} = \frac{TP}{TP + FN}$$

- F1-Score (Dice Coefficient): Harmonic mean of Precision and Recall. It provides a balanced measure of the model's performance, especially when there is an uneven class distribution between foreground and background.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}$$

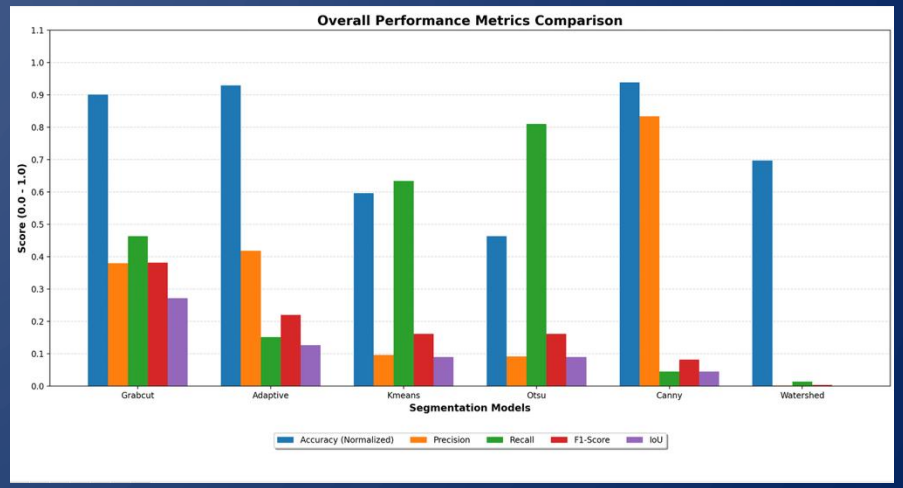
- Mean Squared Error (MSE): Calculates the average of the squares of the errors between the predicted mask (P) and the ground truth (G).It represents the cumulative squared error at the pixel level.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(G_i - P_i)^2$$

QUANTITATIVE RESULTS & COMPARATIVE ANALYSIS

** These metrics represent the **average performance across the entire dataset (6 images)**, providing a generalized comparative analysis of each segmentation model.*

METRICS / MODELS	Accuracy	Precision	Recall	F1-Score	IoU	MSE
Otsu	46.30	0.09	0.81	0.16	0.09	34920.57
Adaptive	92.87	0.42	0.15	0.22	0.13	4638.95
Canny	93.86	0.83	0.05	0.08	0.05	3990.10
K-Means	59.59	0.10	0.63	0.16	0.09	26276.74
GrabCut	90.11	0.38	0.46	0.38	0.27	6428.48
Watershed	69.73	0.00	0.01	0.00	0.00	19684.45



- **Best Overall Performer:** **GrabCut** achieved the highest **IoU (0.27)** and **F1-Score (0.38)**, effectively balancing precision and recall for complex foreground extraction.
- **Thresholding Trade-offs:** **Otsu's Method** reached the maximum **Recall (0.81)** but suffered from low precision due to shadow-induced noise.
- **Accuracy Paradox:** **Canny** shows the highest **Accuracy (93.86%)** because it correctly identifies the background, but its near-zero **IoU (0.05)** highlights its failure to segment the object's body in low-contrast conditions.
- **Lowest Error Rate:** **Canny Edge Detection** achieved the minimum **MSE (3990.10)**, proving its high fidelity in background reconstruction despite poor object segmentation.
- **Adaptive Advantage:** **Adaptive Thresholding** outperformed **Otsu** in **Precision (0.42)** and **Accuracy (92.87%)**, demonstrating better resilience to non-uniform lighting and shadows.

KEY FINDINGS

- ✓ **Traditional vs. Modern:** Traditional segmentation methods vary significantly in performance based on image contrast and background complexity.
- ✓ **The Best Model:** **GrabCut** emerged as the most reliable model for e-commerce-style product extraction, achieving a superior balance between **Precision** and **Recall**.
- ✓ **Metric Importance:** Higher **Pixel Accuracy** can be misleading; metrics like **IoU** and **F1-Score** are more critical for evaluating object extraction quality.

FUTURE WORK

- ❖ **Deep Learning:** Implementing modern architectures like **U-Net** or **Mask R-CNN** for fully automated semantic segmentation.
- ❖ **Hybrid Pipelines:** Combining **Edge-Based** and **Region-Based** methods to refine boundaries in low-contrast environments.
- ❖ **Real-time Processing:** Optimizing the **Python/OpenCV** pipeline for real-time background removal in video streams.

📌 ***Final Insight:*** *Effective pre-processing and model selection are the most important parts of accurate automated background removal.*

THANK YOU FOR LISTENING TO US!

Hüseyin KAYA - 220303036
Ada Şevval SARİ - 220303023